

```
In [1]: import pandas as pd
import numpy as np
df = pd.read_csv("C:/Users/DELL/Downloads/bank_transactions.csv")
df
```

```
Out[1]:
```

	transaction_id	customer_id	merchant_id	transaction_time	amount	country	
0	TXN1036024	1593	10	2025-01-01 00:01:26	4210.27	Singapore	Sin
1	TXN1024284	1885	51	2025-01-01 00:02:00	2302.81	India	M
2	TXN1030530	1606	297	2025-01-01 00:03:38	1422.20	India	
3	TXN1008220	752	244	2025-01-01 00:04:05	932.64	India	Hyc
4	TXN1015332	1974	68	2025-01-01 00:05:10	1723.17	India	M
...	...	...	...	...	...	...	
49995	TXN1003593	488	290	2025-01-31 23:56:56	788.88	UAE	Abi
49996	TXN1036168	1325	112	2025-01-31 23:57:01	127.70	India	
49997	TXN1017995	1422	116	2025-01-31 23:57:50	1022.93	India	Hyc
49998	TXN1010857	1935	196	2025-01-31 23:58:50	1149.18	India	
49999	TXN1049937	710	155	2025-01-31 23:59:00	1091.77	India	Ba

50000 rows × 11 columns



```
In [3]: df.shape
```

```
Out[3]: (50000, 11)
```

```
In [4]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   transaction_id         50000 non-null  object
1   customer_id            50000 non-null  int64
2   merchant_id            50000 non-null  int64
3   transaction_time        50000 non-null  object
4   amount                 50000 non-null  float64
5   country                50000 non-null  object
6   city                   50000 non-null  object
7   payment_method         50000 non-null  object
8   status                 50000 non-null  object
9   device_id              50000 non-null  int64
10  ip_address             50000 non-null  object
dtypes: float64(1), int64(3), object(7)
memory usage: 4.2+ MB

```

```

In [10]: df["transaction_time"] = pd.to_datetime(df["transaction_time"])
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   transaction_id         50000 non-null  object
1   customer_id            50000 non-null  int64
2   merchant_id            50000 non-null  int64
3   transaction_time        50000 non-null  datetime64[ns]
4   amount                 50000 non-null  float64
5   country                50000 non-null  object
6   city                   50000 non-null  object
7   payment_method         50000 non-null  object
8   status                 50000 non-null  object
9   device_id              50000 non-null  int64
10  ip_address             50000 non-null  object
dtypes: datetime64[ns](1), float64(1), int64(3), object(6)
memory usage: 4.2+ MB

```

FRAUD RULE 1: High Amount

```

In [11]: df["flag_high_amount"] = (df["amount"] >= 50000).astype(int)

df["flag_high_amount"].value_counts()

```

```

Out[11]: flag_high_amount
0      49517
1        483
Name: count, dtype: int64

```



FRAUD RULE 2: Velocity Fraud (4+ txns in 10 min)

```
In [12]: df = df.sort_values(["customer_id", "transaction_time"]).reset_index(drop=True)

df["txn_count_10min"] = (
    df.groupby("customer_id", group_keys=False)
      .apply(lambda g: g.set_index("transaction_time")["transaction_id"].rolling("10min")
        .reset_index(drop=True))
)

df["flag_velocity_10min"] = (df["txn_count_10min"] >= 4).astype(int)

df["flag_velocity_10min"].value_counts()
```

C:\Users\DELL\AppData\Local\Temp\ipykernel_3392\815651746.py:5: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
.apply(lambda g: g.set_index("transaction_time")["transaction_id"].rolling("10min").count())
```

```
Out[12]: flag_velocity_10min
0      49727
1        273
Name: count, dtype: int64
```

FRAUD RULE 3: Same Merchant Spike (within 5 min)

```
In [13]: df = df.sort_values(["customer_id", "merchant_id", "transaction_time"]).reset_index()

df["prev_txn_time_same_merchant"] = df.groupby(
    ["customer_id", "merchant_id"]
  )["transaction_time"].shift(1)

df["time_diff_min_same_merchant"] = (
    (df["transaction_time"] - df["prev_txn_time_same_merchant"]).dt.total_seconds()
)

df["flag_same_merchant_5min"] = (
    (df["time_diff_min_same_merchant"] <= 5)
    & (df["time_diff_min_same_merchant"].notna())
).astype(int)

df["flag_same_merchant_5min"].value_counts()
```

```
Out[13]: flag_same_merchant_5min
0      49998
1         2
Name: count, dtype: int64
```

FRAUD RULE 4: Odd Hour (12AM–5AM)

```
In [14]: df["flag_odd_hour"] = (
    (df["transaction_time"].dt.hour >= 0) &
    (df["transaction_time"].dt.hour < 5)
).astype(int)

df["flag_odd_hour"].value_counts()
```

```
Out[14]: flag_odd_hour
0      38857
1      11143
Name: count, dtype: int64
```

FRAUD RULE 5: Location Change (Home Country vs Current)

```
In [16]: home_country_map = (
    df.groupby("customer_id")["country"]
    .agg(lambda x: x.value_counts().index[0])
)

df["home_country"] = df["customer_id"].map(home_country_map)

df["flag_location_change"] = (df["country"] != df["home_country"]).astype(int)

df["flag_location_change"].value_counts()
```

```
Out[16]: flag_location_change
0      42171
1       7829
Name: count, dtype: int64
```

FRAUD RULE 6: Failed Attempts then Success (15 min)

```
In [17]: df = df.sort_values(["customer_id", "transaction_time"]).reset_index(drop=True)

df["is_failed"] = (df["status"] == "FAILED").astype(int)

df["failed_count_15min"] = (
    df.groupby("customer_id", group_keys=False)
    .apply(lambda g: g.set_index("transaction_time")["is_failed"].rolling("15min"
```

```

        .reset_index(drop=True)
    )

    df["flag_failed_then_success"] = (
        (df["status"] == "SUCCESS") & (df["failed_count_15min"] >= 3)
    ).astype(int)

    df["flag_failed_then_success"].value_counts()

```

C:\Users\DELL\AppData\Local\Temp\ipykernel_3392\3733025018.py:7: FutureWarning: Data FrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```

        .apply(lambda g: g.set_index("transaction_time")["is_failed"].rolling("15min").sum())

```

Out[17]: flag_failed_then_success
0 50000
Name: count, dtype: int64



FRAUD SCORE + RISK LEVEL (Final Output)

```

In [18]: flags = [
    "flag_high_amount",
    "flag_velocity_10min",
    "flag_same_merchant_5min",
    "flag_odd_hour",
    "flag_location_change",
    "flag_failed_then_success"
]

weights = {
    "flag_high_amount": 3,
    "flag_velocity_10min": 2,
    "flag_same_merchant_5min": 2,
    "flag_odd_hour": 1,
    "flag_location_change": 2,
    "flag_failed_then_success": 2
}

df["fraud_score"] = sum(df[col] * w for col, w in weights.items())

```

Risk Level

```

In [19]: df["risk_level"] = np.where(
    df["fraud_score"] >= 5, "High",
    np.where(df["fraud_score"] >= 3, "Medium", "Low")
)

```

```
df["risk_level"].value_counts()
```

```
Out[19]: risk_level
Low      47712
Medium   2224
High      64
Name: count, dtype: int64
```

Fraud Reasons

```
In [20]: df["fraud_reasons"] = df[flags].apply(
        lambda row: ", ".join([col.replace("flag_", "") for col in flags if row[col] ==
                                axis=1
                                ])
        )

df[["transaction_id", "customer_id", "amount", "fraud_score", "risk_level", "fraud_reasons"]]
```

```
Out[20]:
```

	transaction_id	customer_id	amount	fraud_score	risk_level	fraud_reasons
0	TXN1029138	1	1037.95	0	Low	
1	TXN1042523	1	3871.92	1	Low	odd_hour
2	TXN1004244	1	5652.44	0	Low	
3	TXN1027139	1	2608.13	0	Low	
4	TXN1015309	1	679.19	0	Low	
5	TXN1039754	1	2791.48	1	Low	odd_hour
6	TXN1028450	1	2370.82	0	Low	
7	TXN1024120	1	984.60	0	Low	
8	TXN1013344	1	954.92	0	Low	
9	TXN1004948	1	1567.49	0	Low	

Export Final Fraud Alerts CSV

```
In [21]: alerts = df[df["risk_level"].isin(["High", "Medium"])].copy()
alerts.to_csv("fraud_alerts.csv", index=False)

print("✅ fraud_alerts.csv saved successfully!")

✅ fraud_alerts.csv saved successfully!
```

```
In [22]: alerts.shape
```

```
Out[22]: (2288, 26)
```

Create Transaction Date Column

```
In [23]: df["txn_date"] = df["transaction_time"].dt.date
```

Create Suspicious Flag (Medium + High)

```
In [24]: df["is_suspicious"] = df["risk_level"].isin(["Medium", "High"]).astype(int)
```

Daily Summary Table

```
In [25]: daily_fraud = df.groupby("txn_date").agg(
    total_transactions=("transaction_id", "count"),
    suspicious_transactions=("is_suspicious", "sum"),
    total_amount=("amount", "sum"),
    suspicious_amount=("amount", lambda x: x[df.loc[x.index, "is_suspicious"] == 1]
).reset_index()
```

Fraud Rate %

```
In [26]: daily_fraud["fraud_rate_pct"] = (
    daily_fraud["suspicious_transactions"] / daily_fraud["total_transactions"] * 10
).round(2)

daily_fraud.head()
```

```
Out[26]:
```

	txn_date	total_transactions	suspicious_transactions	total_amount	suspicious_amount	f
--	----------	--------------------	-------------------------	--------------	-------------------	---

0	2025-01-01	1592	75	5424338.97	2426357.91	
1	2025-01-02	1632	68	4844793.32	1680983.90	
2	2025-01-03	1538	74	4930505.96	2010086.68	
3	2025-01-04	1600	78	4731972.92	1750405.81	
4	2025-01-05	1623	79	5662653.51	2551260.29	

◀ ————— ▶

Top 10 Highest Fraud Rate Days

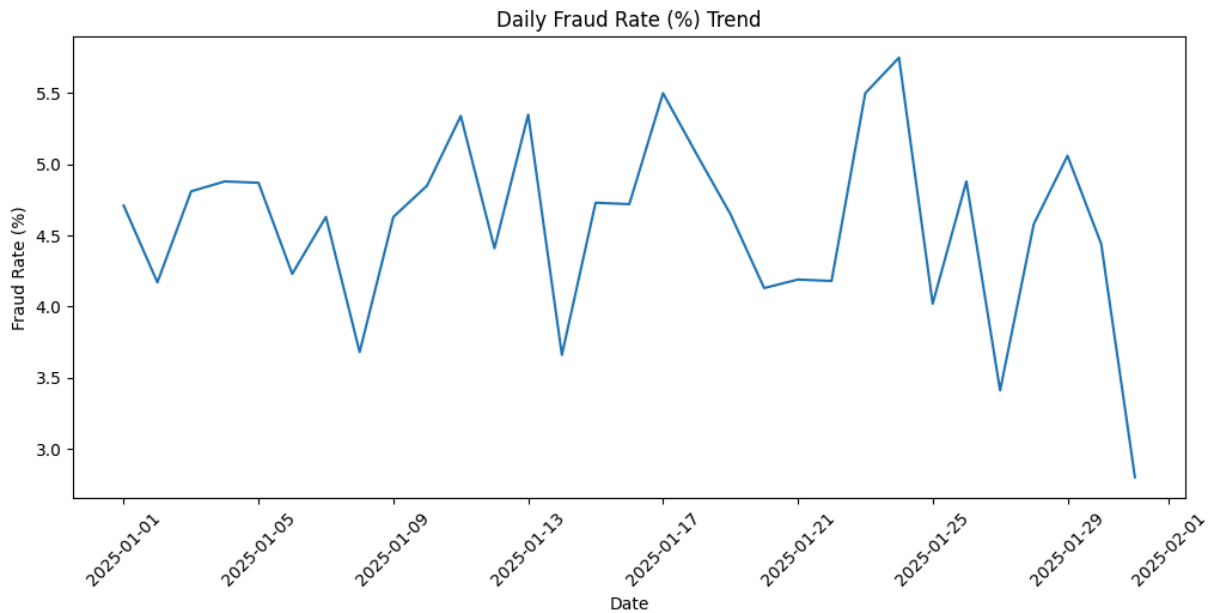
```
In [27]: daily_fraud.to_csv("daily_fraud_trend.csv", index=False)
print("✅ daily_fraud_trend.csv saved successfully!")
```

✅ daily_fraud_trend.csv saved successfully!

Plot Fraud Rate Trend

```
In [28]: import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))
plt.plot(daily_fraud["txn_date"], daily_fraud["fraud_rate_pct"])
plt.xticks(rotation=45)
plt.title("Daily Fraud Rate (%) Trend")
plt.xlabel("Date")
plt.ylabel("Fraud Rate (%)")
plt.show()
```



```
In [29]: daily_fraud.head()
```


Out[29]:

	txn_date	total_transactions	suspicious_transactions	total_amount	suspicious_amount	f
0	2025-01-01	1592	75	5424338.97	2426357.91	
1	2025-01-02	1632	68	4844793.32	1680983.90	
2	2025-01-03	1538	74	4930505.96	2010086.68	
3	2025-01-04	1600	78	4731972.92	1750405.81	
4	2025-01-05	1623	79	5662653.51	2551260.29	

In [30]: `daily_fraud.sort_values("fraud_rate_pct", ascending=False).head(5)`

Out[30]:

	txn_date	total_transactions	suspicious_transactions	total_amount	suspicious_amount
23	2025-01-24	1617	93	5810345.03	2823701.51
22	2025-01-23	1637	90	5802301.03	2728119.22
16	2025-01-17	1599	88	4613445.22	1691776.14
12	2025-01-13	1644	88	4817295.23	1944294.84
10	2025-01-11	1628	87	5580516.36	2596398.18

Top Suspicious Customers + Merchants + Summary Insights 🚨

In [31]:

```

top_customers = df[df["is_suspicious"] == 1].groupby("customer_id").agg(
    suspicious_txn_count=("transaction_id", "count"),
    total_suspicious_amount=("amount", "sum"),
    avg_suspicious_amount=("amount", "mean"),
    max_fraud_score=("fraud_score", "max")
).reset_index()

top_customers = top_customers.sort_values("suspicious_txn_count", ascending=False)
top_customers.head(10)

```

Out[31]:

	customer_id	suspicious_txn_count	total_suspicious_amount	avg_suspicious_amount
1191	1788	7	25818.95	3688.421429
566	837	7	20895.92	2985.131429
668	998	6	409543.39	68257.231667
250	382	6	13353.24	2225.540000
848	1280	6	8156.52	1359.420000
1032	1549	6	10748.29	1791.381667
850	1283	6	6530.76	1088.460000
122	186	6	7540.38	1256.730000
16	28	6	12057.59	2009.598333
409	612	5	171064.93	34212.986000

In [32]: `top_customers.to_csv("top_suspicious_customers.csv", index=False)`
`print("✅ top_suspicious_customers.csv saved!")`

✅ top_suspicious_customers.csv saved!

Top Suspicious Merchants (by count)

In [33]: `top_merchants = df[df["is_suspicious"] == 1].groupby("merchant_id").agg(
 suspicious_txn_count=("transaction_id", "count"),
 total_suspicious_amount=("amount", "sum"),
 avg_suspicious_amount=("amount", "mean"),
 max_fraud_score=("fraud_score", "max")
).reset_index()`

`top_merchants = top_merchants.sort_values("suspicious_txn_count", ascending=False)`
`top_merchants.head(10)`

Out[33]:

	merchant_id	suspicious_txn_count	total_suspicious_amount	avg_suspicious_amount	n
287	288	18	481977.48	26776.526667	
257	258	16	274664.24	17166.515000	
229	230	15	321411.22	21427.414667	
51	52	15	940246.39	62683.092667	
4	5	14	335850.20	23989.300000	
72	73	14	277180.95	19798.639286	
252	253	14	166983.36	11927.382857	
297	298	14	487491.05	34820.789286	
5	6	14	388545.46	27753.247143	
80	81	13	221052.45	17004.034615	

◀  ▶

Breakdown of Fraud Reasons (Most Triggered Rules)

```
In [34]: reason_counts = df[flags].sum().sort_values(ascending=False).reset_index()
reason_counts.columns = ["fraud_rule", "trigger_count"]
reason_counts
```

Out[34]:

	fraud_rule	trigger_count
0	flag_odd_hour	11143
1	flag_location_change	7829
2	flag_high_amount	483
3	flag_velocity_10min	273
4	flag_same_merchant_5min	2
5	flag_failed_then_success	0

```
In [35]: reason_counts.to_csv("fraud_rule_trigger_summary.csv", index=False)
print("✅ fraud_rule_trigger_summary.csv saved!")
```

✅ fraud_rule_trigger_summary.csv saved!

Show Top 20 High-Risk Transactions (Investigation View)

```
In [36]: high_risk_cases = df[df["risk_level"] == "High"].sort_values("fraud_score", ascending=True,
    ["transaction_id", "customer_id", "merchant_id", "transaction_time", "amount",
     "country", "status", "fraud_score", "risk_level", "fraud_reasons"]
    ]

high_risk_cases.head(20)
```

Out[36]:

	transaction_id	customer_id	merchant_id	transaction_time	amount	country	
45427	TXN1004053	1820	100	2025-01-12 02:47:21	72970.70	USA	SI
11184	TXN1046662	460	153	2025-01-10 04:42:58	134360.08	USA	SI
19506	TXN1045971	794	219	2025-01-05 01:07:01	139718.09	Singapore	SI
3895	TXN1022479	161	192	2025-01-16 02:18:19	63075.03	USA	
25016	TXN1031169	1008	82	2025-01-27 03:25:31	84356.98	UAE	SI
18669	TXN1049963	759	251	2025-01-27 04:01:56	194015.80	USA	SI
25887	TXN1001684	1043	62	2025-01-31 01:49:15	118143.31	USA	SI
18162	TXN1015233	739	12	2025-01-16 00:07:19	148054.74	UAE	SI
26663	TXN1043237	1074	191	2025-01-12 04:26:55	73942.90	USA	SI
31446	TXN1006264	1263	168	2025-01-16 04:40:34	125605.09	UK	SI
31987	TXN1021485	1284	218	2025-01-23 03:37:33	131054.04	UK	SI
14020	TXN1024845	571	298	2025-01-24 01:17:02	82268.24	Singapore	SI
21612	TXN1034507	876	71	2025-01-13 04:50:06	133166.58	Singapore	SI
34580	TXN1002629	1388	62	2025-01-22 01:05:00	65434.02	USA	SI
13866	TXN1038585	565	93	2025-01-21 03:01:38	147997.17	Singapore	SI
20279	TXN1012613	824	269	2025-01-26 02:03:56	113212.05	USA	SI
4519	TXN1016932	187	298	2025-01-24 02:12:48	198315.79	USA	SI
36329	TXN1045826	1459	161	2025-01-23 01:39:45	176720.89	UK	SI
8165	TXN1005920	338	142	2025-01-17 02:22:31	168123.18	USA	SI

	transaction_id	customer_id	merchant_id	transaction_time	amount	country	
5639	TXN1004417	232	184	2025-01-21 02:34:55	166772.73	UK	SI

```
In [37]: high_risk_cases.to_csv("high_risk_cases.csv", index=False)
print("✅ high_risk_cases.csv saved!")
```

✅ high_risk_cases.csv saved!

```
In [39]: top_customers.head(5)
```

```
Out[39]:
```

	customer_id	suspicious_txn_count	total_suspicious_amount	avg_suspicious_amount	
1191	1788	7	25818.95	3688.421429	
566	837	7	20895.92	2985.131429	
668	998	6	409543.39	68257.231667	
250	382	6	13353.24	2225.540000	
848	1280	6	8156.52	1359.420000	

◀  ▶

```
In [40]: top_merchants.head(5)
```

```
Out[40]:
```

	merchant_id	suspicious_txn_count	total_suspicious_amount	avg_suspicious_amount	n
287	288	18	481977.48	26776.526667	
257	258	16	274664.24	17166.515000	
229	230	15	321411.22	21427.414667	
51	52	15	940246.39	62683.092667	
4	5	14	335850.20	23989.300000	

◀  ▶

```
In [41]: reason_counts
```

```
Out[41]:
```

	fraud_rule	trigger_count
0	flag_odd_hour	11143
1	flag_location_change	7829
2	flag_high_amount	483
3	flag_velocity_10min	273
4	flag_same_merchant_5min	2
5	flag_failed_then_success	0

